Chapter 7: Sequential Designs

Youngseo Cho

2025-12-31

Contents

1 Introduction **1**

2 Section 7.1: Emulation **1**
2.1 7.1.1 Prediction-based Sequential Design 1

3 Section 7.2: Optimization **8**
3.1 Expected Improvement for Optimization 8

4 Section 7.2: Optimization Strategies **10**

5 Section 7.3: Inverse Designs (OSFD) **10**

6 Summary **11**

1 Introduction

This chapter covers sequential design strategies for emulation and optimization.

Citation: V. R. Joseph (2026). *Experimental Design for Data Science and Engineering.* CRC Press.

```
library(rkriging)
library(MaxPro)
library(spacefillr)
library(FNN)
library(pdist)
```

2 Section 7.1: Emulation

2.1 7.1.1 Prediction-based Sequential Design

2.1.1 Branin Function

```
f <- function(x) {
  x1 <- x[1] * 15 - 5
  x2 <- x[2] * 15
  val1 <- (x2 - 5.1/(4 * pi^2) * (x1^2) + 5/pi * x1 - 6)^2
  val2 <- 10 * (1 - 1/(8 * pi)) * cos(x1) + 10
  return(val1 + val2)
}
```

2.1.2 ALC (Active Learning Cohn)

```
ALC <- function(D0, y0, Nnew) {
  D <- D0
  y <- y0
  n <- nrow(D)
  p <- ncol(D)
  test <- generate_sobol_set(1000*p, p, seed = sample(10^6, 1))
  truey <- apply(test, 1, f)
  rmse <- numeric(Nnew + 1)

  for (i in 1:Nnew) {
    CAND0 <- generate_sobol_set(100*p, p, seed = sample(10^6, 1))
    if (i == 1) theta <- rep(1, p)
    kernel <- Gaussian.Kernel(theta)
    obj1 <- Fit.Kriging(D, y, model = "OK", kernel = kernel)
    obj2 <- Fit.Kriging(D, y, kernel.parameters = list(type = "Gaussian"))
    if (obj1$get_nllh() < obj2$get_nllh()) obj <- obj1 else obj <- obj2
    theta <- obj$get_lengthscale()
    pred <- Predict.Kriging(obj, test)$mean
    rmse[i] <- sqrt(mean((pred - truey)^2))

    kernel <- Gaussian.Kernel(theta)
    R <- Evaluate.Kernel(kernel, D)
    Rinv <- solve(R + 10^(-6) * diag(n))

    r <- function(x, theta) {
      if (p > 1) {
        A <- D - rep(1, n) %*% t(x)
        g <- exp(-0.5 * apply(abs(A %*% diag(1/theta))^2, 1, sum))
      } else {
        g <- exp(-0.5 * abs((D - x)/theta)^2)
      }
      return(g)
    }
    basis <- function(h, theta) exp(-0.5 * sum((h/theta)^2))

    opt <- function(v, theta) {
      a <- r(v, theta)
      b <- c(Rinv %*% a)
      numer <- function(x, v, theta) {
        val <- (basis(x - v, theta) - sum(b * r(x, theta)))^2
        return(val)
      }
      val <- mean(apply(CAND0, 1, numer, v = v, theta = theta)) / max(1 - sum(a * b), 10^(-6))
      return(val)
    }

    d <- c(knn.dist(D, k = 1)) / 2
    dis <- as.matrix(pdist(D, CAND0))
    N <- dim(CAND0)[1]
    ind <- NULL
    for (j in 1:n) ind <- c(ind, (1:N)[dis[j, ] < d[j]])
    if (length(ind) > 0) CAND <- CAND0[-ind, ] else CAND <- CAND0
```

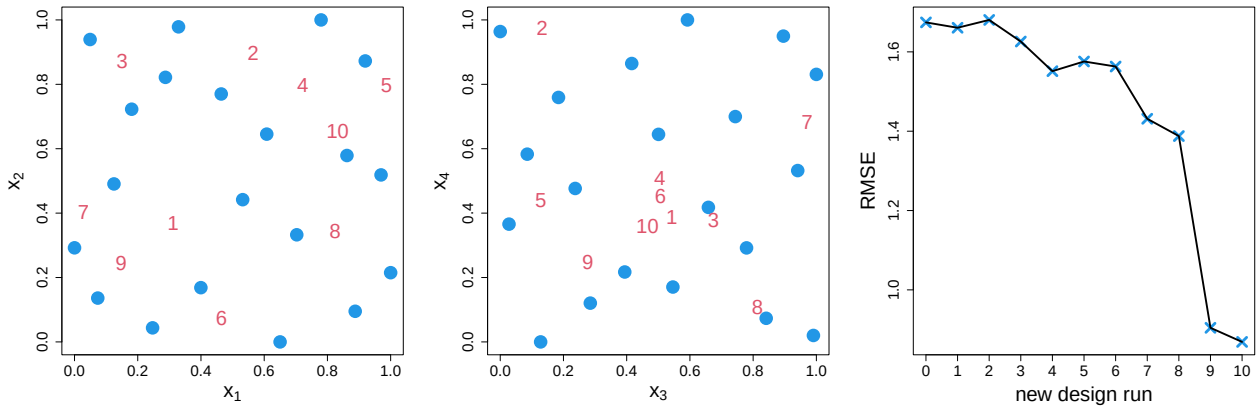
```
    xnew <- CAND[which.max(apply(CAND, 1, opt, theta = theta)), ]
    D <- rbind(D, xnew)
    y <- c(y, f(xnew))
    n <- n + 1
  }

  kernel <- Gaussian.Kernel(theta)
  obj1 <- Fit.Kriging(D, y, model = "OK", kernel = kernel)
  obj2 <- Fit.Kriging(D, y, kernel.parameters = list(type = "Gaussian"))
  if (obj1$get_nllh() < obj2$get_nllh()) obj <- obj1 else obj <- obj2
  pred <- Predict.Kriging(obj, test)$mean
  rmse[Nnew + 1] <- sqrt(mean((pred - truey)^2))

  return(list("D" = D, "rmse" = rmse))
}

set.seed(1)
p <- 4
n <- 20
D0 <- MaxPro(MaxProLHD(n, p)$Design)$Design
y0 <- apply(D0, 1, f)
a <- ALC(D0, y0, Nnew = 10)
D <- a$D
rmse <- a$rmse

par(mar = c(5, 5, 4, 2))
par(mfrow = c(1, 3))
plot(D[1:n, 1:2], xlim = c(0, 1), ylim = c(0, 1),
     xlab = expression(x[1]), ylab = expression(x[2]),
     cex.lab = 2, cex.axis = 1.5, pch = 16, col = 4, cex = 3)
text(D[(n+1):(n+10), 1:2], col = 2, labels = 1:10, cex = 2)
plot(D[1:n, 3:4], xlim = c(0, 1), ylim = c(0, 1),
     xlab = expression(x[3]), ylab = expression(x[4]),
     cex.lab = 2, cex.axis = 1.5, pch = 16, col = 4, cex = 3)
text(D[(n+1):(n+10), 3:4], col = 2, labels = 1:10, cex = 2)
plot(1:11, rmse, pch = 4, xaxt = "n", xlab = "new design run", ylab = "RMSE",
     cex.lab = 2, cex.axis = 1.5, col = 4, cex = 2, lwd = 3)
lines(1:11, rmse, lwd = 2)
axis(1, at = 1:11, labels = 0:10, cex.axis = 1.5)
```



```r
par(mfrow = c(1, 1))
```

2.1.3 ALMV (Active Learning MacKay-Vivarelli)

```r
ALMV <- function(D0, y0, Nnew) {
  D <- D0
  y <- y0
  n <- nrow(D)
  p <- ncol(D)
  test <- generate_sobol_set(1000*p, p, seed = sample(10^6, 1))
  truey <- apply(test, 1, f)
  rmse <- numeric(Nnew + 1)

  for (i in 1:Nnew) {
    CAND0 <- generate_sobol_set(100*p, p, seed = sample(10^6, 1))
    if (i == 1) theta <- rep(1, p)
    kernel <- Gaussian.Kernel(theta)
    obj1 <- Fit.Kriging(D, y, model = "OK", kernel = kernel)
    obj2 <- Fit.Kriging(D, y, kernel.parameters = list(type = "Gaussian"))
    if (obj1$get_nllh() < obj2$get_nllh()) obj <- obj1 else obj <- obj2
    theta <- obj$get_lengthscale()
    pred <- Predict.Kriging(obj, test)$mean
    rmse[i] <- sqrt(mean((pred - truey)^2))

    kernel <- Gaussian.Kernel(theta)
    R <- Evaluate.Kernel(kernel, D)
    U <- chol(R + 10^(-6) * diag(n))

    r <- function(x, theta) {
      if (p > 1) {
        A <- D - rep(1, n) %*% t(x)
        g <- exp(-0.5 * apply(abs(A %*% diag(1/theta))^2, 1, sum))
      } else {
        g <- exp(-0.5 * abs((D - x)/theta)^2)
      }
      return(g)
    }
    basis <- function(h, theta) exp(-0.5 * sum((h/theta)^2))

    opt <- function(v, theta) {
      b <- forwardsolve(t(U), r(v, theta))
      numer <- function(x, v, theta) {
        a <- forwardsolve(t(U), r(x, theta))
        val <- 1 - sum(a^2) - (basis(x - v, theta) - sum(a * b))^2 / max(1 - sum(b^2), 10^(-6))
        return(val)
      }
      val <- max(apply(CAND0, 1, numer, v = v, theta = theta)) / max(1 - sum(b^2), 10^(-6))
      return(val)
    }

    d <- c(knn.dist(D, k = 1)) / 2
    dis <- as.matrix(pdist(D, CAND0))
    N <- dim(CAND0)[1]
```

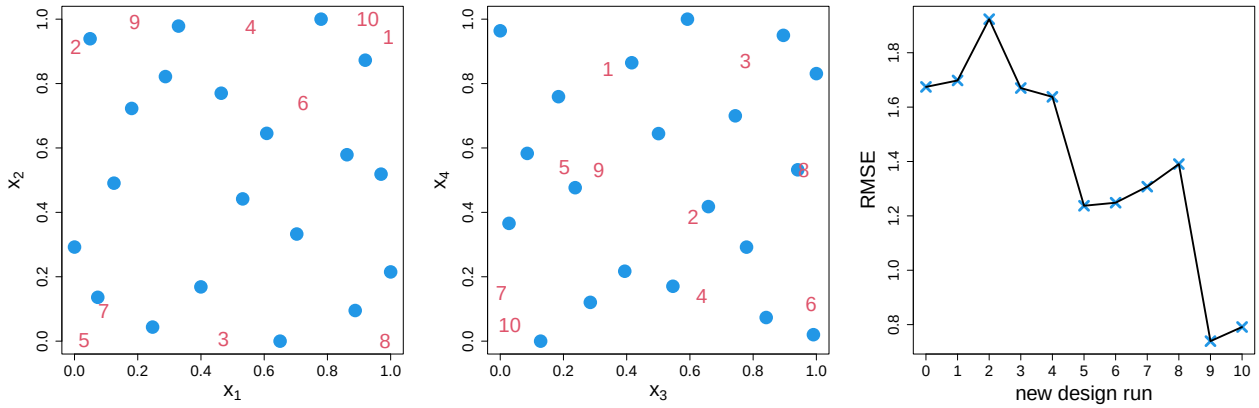
```
    ind <- NULL
    for (j in 1:n) ind <- c(ind, (1:N)[dis[j, ] < d[j]])
    if (length(ind) > 0) CAND <- CAND0[-ind, ] else CAND <- CAND0
    xnew <- CAND[which.min(apply(CAND, 1, opt, theta = theta)), ]
    D <- rbind(D, xnew)
    y <- c(y, f(xnew))
    n <- n + 1
  }

  kernel <- Gaussian.Kernel(theta)
  obj1 <- Fit.Kriging(D, y, model = "OK", kernel = kernel)
  obj2 <- Fit.Kriging(D, y, kernel.parameters = list(type = "Gaussian"))
  if (obj1$get_nllh() < obj2$get_nllh()) obj <- obj1 else obj <- obj2
  pred <- Predict.Kriging(obj, test)$mean
  rmse[Nnew + 1] <- sqrt(mean((pred - truey)^2))

  return(list("D" = D, "rmse" = rmse))
}

set.seed(1)
p <- 4
n <- 20
D0 <- MaxPro(MaxProLHD(n, p)$Design)$Design
y0 <- apply(D0, 1, f)
a <- ALMV(D0, y0, Nnew = 10)
D <- a$D
rmse <- a$rmse

par(mar = c(5, 5, 4, 2))
par(mfrow = c(1, 3))
plot(D[1:n, 1:2], xlim = c(0, 1), ylim = c(0, 1),
     xlab = expression(x[1]), ylab = expression(x[2]),
     cex.lab = 2, cex.axis = 1.5, pch = 16, col = 4, cex = 3)
text(D[(n+1):(n+10), 1:2], col = 2, labels = 1:10, cex = 2)
plot(D[1:n, 3:4], xlim = c(0, 1), ylim = c(0, 1),
     xlab = expression(x[3]), ylab = expression(x[4]),
     cex.lab = 2, cex.axis = 1.5, pch = 16, col = 4, cex = 3)
text(D[(n+1):(n+10), 3:4], col = 2, labels = 1:10, cex = 2)
plot(1:11, rmse, pch = 4, xaxt = "n", xlab = "new design run", ylab = "RMSE",
     cex.lab = 2, cex.axis = 1.5, col = 4, cex = 2, lwd = 3)
lines(1:11, rmse, lwd = 2)
axis(1, at = 1:11, labels = 0:10, cex.axis = 1.5)
```




```
par(mfrow = c(1, 1))
```

2.1.4 ALM (Active Learning MacKay)

```
ALM <- function(D0, y0, Nnew) {
  D <- D0
  y <- y0
  n <- nrow(D)
  p <- ncol(D)
  test <- generate_sobol_set(1000*p, p, seed = sample(10^6, 1))
  truey <- apply(test, 1, f)
  rmse <- numeric(Nnew + 1)

  for (i in 1:Nnew) {
    CAND0 <- generate_sobol_set(1000*p, p, seed = sample(10^6, 1))
    if (i == 1) theta <- rep(1, p)
    kernel <- Gaussian.Kernel(theta)
    obj1 <- Fit.Kriging(D, y, model = "OK", kernel = kernel)
    obj2 <- Fit.Kriging(D, y, kernel.parameters = list(type = "Gaussian"))
    if (obj1$get_nllh() < obj2$get_nllh()) obj <- obj1 else obj <- obj2
    theta <- obj$get_lengthscale()
    pred <- Predict.Kriging(obj, test)$mean
    rmse[i] <- sqrt(mean((pred - truey)^2))

    kernel <- Gaussian.Kernel(theta)
    R <- Evaluate.Kernel(kernel, D)
    Rinv <- solve(R + 10^(-6) * diag(n))

    r <- function(x, theta) {
      if (p > 1) {
        A <- D - rep(1, n) %*% t(x)
        g <- exp(-0.5 * apply(abs(A %*% diag(1/theta))^2, 1, sum))
      } else {
        g <- exp(-0.5 * abs((D - x)/theta)^2)
      }
      return(g)
    }

    opt <- function(v, theta) {
      a <- r(v, theta)
```

```
      b <- c(Rinv %*% a)
      val <- 1 - sum(a * b)
      return(val)
    }

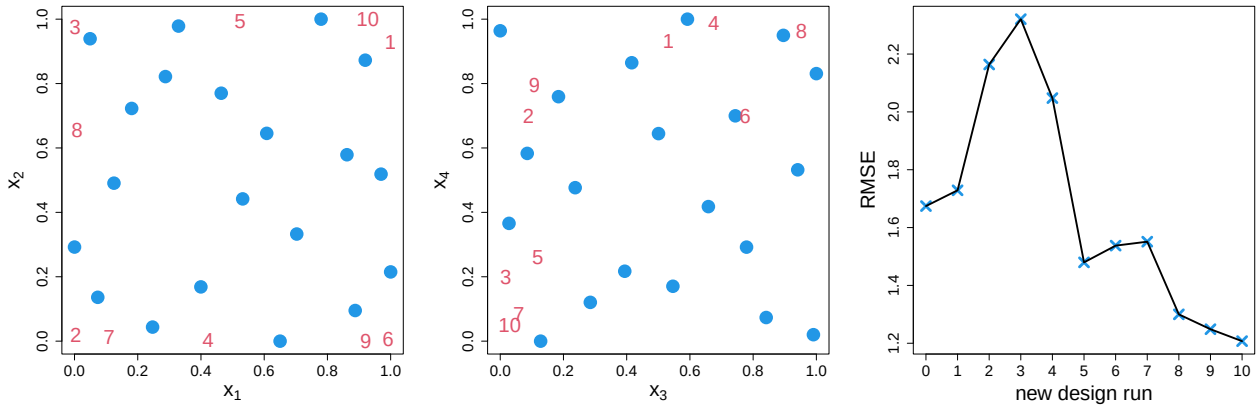
    d <- c(knn.dist(D, k = 1)) / 2
    dis <- as.matrix(pdist(D, CAND0))
    N <- dim(CAND0)[1]
    ind <- NULL
    for (j in 1:n) ind <- c(ind, (1:N)[dis[j, ] < d[j]])
    if (length(ind) > 0) CAND <- CAND0[-ind, ] else CAND <- CAND0
    xnew <- CAND[which.max(apply(CAND, 1, opt, theta = theta)), ]
    D <- rbind(D, xnew)
    y <- c(y, f(xnew))
    n <- n + 1
  }

  kernel <- Gaussian.Kernel(theta)
  obj1 <- Fit.Kriging(D, y, model = "OK", kernel = kernel)
  obj2 <- Fit.Kriging(D, y, kernel.parameters = list(type = "Gaussian"))
  if (obj1$get_nllh() < obj2$get_nllh()) obj <- obj1 else obj <- obj2
  pred <- Predict.Kriging(obj, test)$mean
  rmse[Nnew + 1] <- sqrt(mean((pred - truey)^2))

  return(list("D" = D, "rmse" = rmse))
}

set.seed(1)
p <- 4
n <- 20
D0 <- MaxPro(MaxProLHD(n, p)$Design)$Design
y0 <- apply(D0, 1, f)
a <- ALM(D0, y0, Nnew = 10)
D <- a$D
rmse <- a$rmse

par(mar = c(5, 5, 4, 2))
par(mfrow = c(1, 3))
plot(D[1:n, 1:2], xlim = c(0, 1), ylim = c(0, 1),
     xlab = expression(x[1]), ylab = expression(x[2]),
     cex.lab = 2, cex.axis = 1.5, pch = 16, col = 4, cex = 3)
text(D[(n+1):(n+10), 1:2], col = 2, labels = 1:10, cex = 2)
plot(D[1:n, 3:4], xlim = c(0, 1), ylim = c(0, 1),
     xlab = expression(x[3]), ylab = expression(x[4]),
     cex.lab = 2, cex.axis = 1.5, pch = 16, col = 4, cex = 3)
text(D[(n+1):(n+10), 3:4], col = 2, labels = 1:10, cex = 2)
plot(1:11, rmse, pch = 4, xaxt = "n", xlab = "new design run", ylab = "RMSE",
     cex.lab = 2, cex.axis = 1.5, col = 4, cex = 2, lwd = 3)
lines(1:11, rmse, lwd = 2)
axis(1, at = 1:11, labels = 0:10, cex.axis = 1.5)
```




```
par(mfrow = c(1, 1))
```

3 Section 7.2: Optimization

3.1 Expected Improvement for Optimization

```
set.seed(1)
f <- function(x) sin(10 * pi * x) / (1 + 64 * (x - 0.25)^2) + x^2

EI <- function(x, hmin, obj) {
  pred <- Predict.Kriging(obj, cbind(x))
  s <- max(pred$sd, 10^(-10))
  u <- (hmin - pred$mean) / s
  val <- s * (u * pnorm(u) + dnorm(u))
  return(val)
}

test <- seq(0, 1, length = 301)
true <- f(test)
n <- 8
D <- ((1:n) - 0.5) / n
y <- f(D)

a <- Fit.Kriging(D, y, kernel.parameters = list(type = "Gaussian"))
pred <- Predict.Kriging(a, test)
low <- pred$mean - 2 * pred$sd
up <- pred$mean + 2 * pred$sd

par(mar = c(5, 4, 5, 4))
par(mfrow = c(1, 3))

# Initial
plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 3, col = 3,
     ylim = c(min(true) - 0.25, max(true) + 0.5), main = "Initial",
     cex.main = 2.5, cex.axis = 1.5, cex.lab = 1.5)
polygon(c(test, rev(test)), c(low, rev(up)), col = "lightgray", border = NA)
lines(test, pred$mean, col = 1, lwd = 4, lty = 2)
points(cbind(D, y), pch = 16, cex = 3, col = 4)
lines(test, true, lty = 3, lwd = 4, col = 3)
legend("topleft", legend = c("truth", "prediction", "EI"),
```

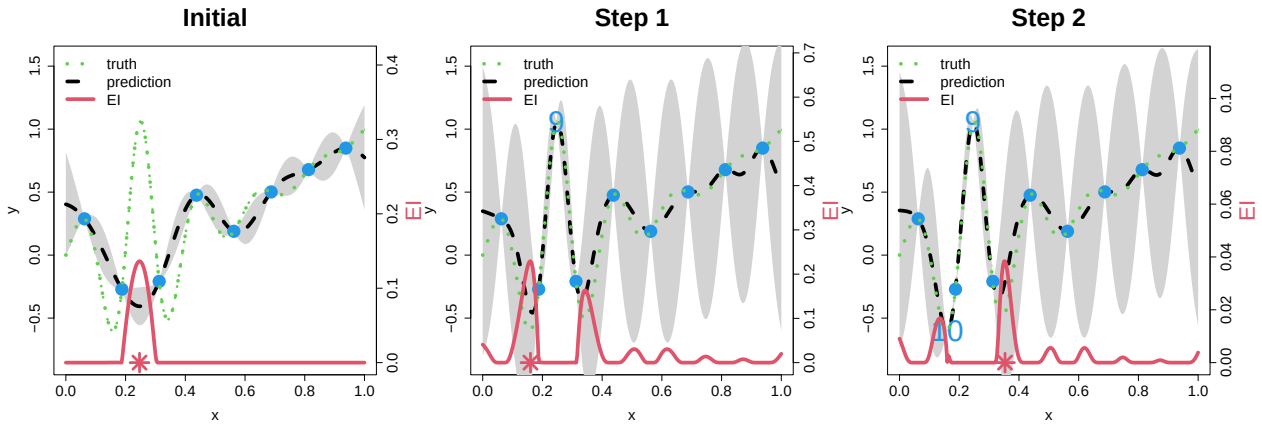
```
       lty = c(3, 2, 1), col = c(3, 1, 2), lwd = c(4, 4, 4), bty = "n", cex = 1.5)

ei <- apply(cbind(test), 1, EI, hmin = min(y), obj = a)
par(new = TRUE)
plot(test, ei, "l", col = 2, lwd = 4, axes = FALSE,
     ylim = c(0, 3 * max(ei)), xlab = "", ylab = "")
axis(side = 4, at = pretty(range(3 * ei)), cex.axis = 1.5)
mtext("EI", side = 4, line = 3, cex = 1.5, col = 2)
xnew <- test[which.max(ei)]
points(xnew, 0, col = 2, cex = 3, pch = 8, lwd = 3)

# Sequential steps
for (i in 1:2) {
  D <- c(D, xnew)
  y <- c(y, f(xnew))
  a <- Fit.Kriging(D, y, kernel.parameters = list(type = "Gaussian"))
  pred <- Predict.Kriging(a, test)
  low <- pred$mean - 2 * pred$sd
  up <- pred$mean + 2 * pred$sd

  par(mar = c(5, 4, 5, 4))
  plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 3, col = 3,
       ylim = c(min(true) - 0.25, max(true) + 0.5), main = paste("Step", i),
       cex.main = 2.5, cex.axis = 1.5, cex.lab = 1.5)
  polygon(c(test, rev(test)), c(low, rev(up)), col = "lightgray", border = NA)
  lines(test, pred$mean, col = 1, lwd = 4, lty = 2)
  points(cbind(D[1:n], y[1:n]), pch = 16, cex = 3, col = 4)
  text(cbind(D[(n+1):(n+i)], y[(n+1):(n+i)]), labels = (n+1):(n+i), col = 4, cex = 3)
  lines(test, true, lty = 3, lwd = 4, col = 3)
  legend("topleft", legend = c("truth", "prediction", "EI"),
         lty = c(3, 2, 1), col = c(3, 1, 2), lwd = c(4, 4, 4), bty = "n", cex = 1.5)

  ei <- apply(cbind(test), 1, EI, hmin = min(y), obj = a)
  par(new = TRUE)
  plot(test, ei, "l", col = 2, lwd = 4, axes = FALSE,
       ylim = c(0, 3 * max(ei)), xlab = "", ylab = "")
  axis(side = 4, at = pretty(range(3 * ei)), cex.axis = 1.5)
  mtext("EI", side = 4, line = 3, cex = 1.5, col = 2)
  xnew <- test[which.max(ei)]
  points(xnew, 0, col = 2, cex = 3, pch = 8, lwd = 3)
}
```




```
par(mfrow = c(1, 1))
```

4 Section 7.2: Optimization Strategies

Sequential optimization using Bayesian optimization with Expected Improvement (EI) balances exploration and exploitation:

- **Exploration**: Sampling in regions with high uncertainty
- **Exploitation**: Sampling near current best solutions

The EI criterion combines both by considering the expected improvement over the current best observation.

5 Section 7.3: Inverse Designs (OSFD)

```
library(OSFD)

f <- function(x) {
  y1 <- 1 / (x[1]^2 + x[2]^2 + 0.01)^(1/2)
  if (x[2] == 0) {
    y2 <- 0
  } else if (x[1] == 0) {
    y2 <- pi / 2
  } else {
    y2 <- atan(x[2] / x[1])
  }
  return(c(y1 = y1, y2 = y2))
}

set.seed(1)
p <- 2
q <- 2
D0 <- MaxProLHD(10, p)$Design
n <- 50
osfd <- OSFD(D = D0, f = f, p = p, q = q, n = n, method = "Greedy")
```

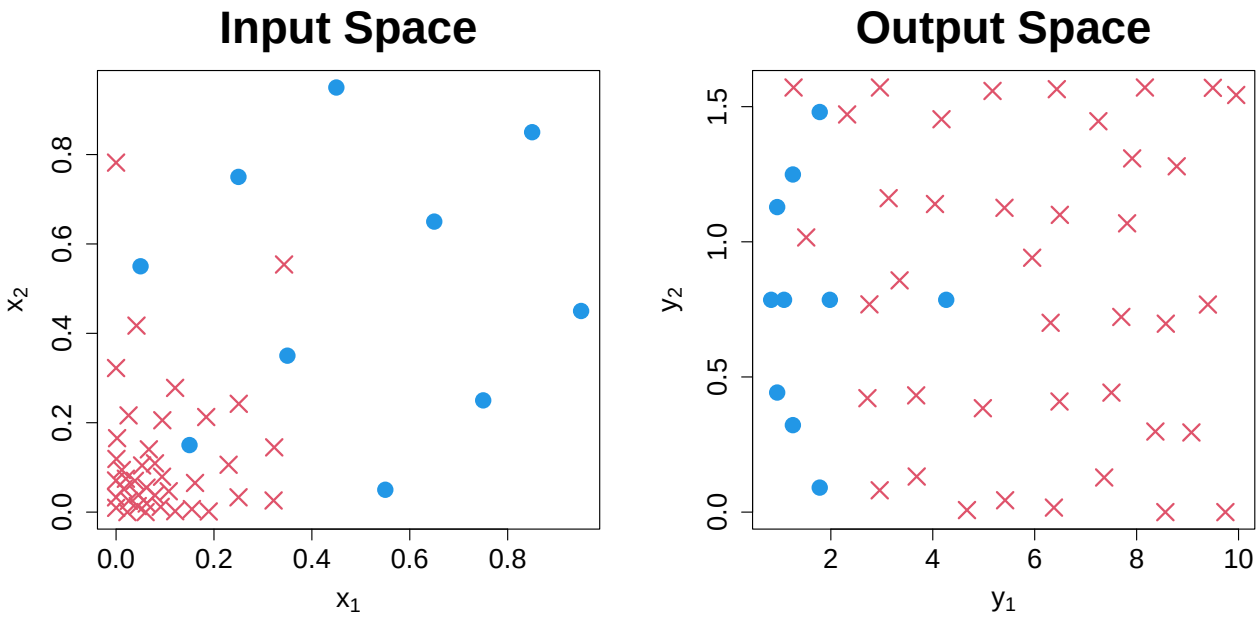
```
##   |                                                                      |
```

```
D <- osfd$D
Y <- osfd$Y
```

```
par(mar = c(5, 5, 4, 2))
par(mfrow = c(1, 2))

plot(D, pch = NA, main = "Input Space",
     xlab = expression(x[1]), ylab = expression(x[2]),
     cex.main = 2.5, cex.axis = 1.5, cex.lab = 1.5)
points(D[1:10, ], pch = 16, col = 4, cex = 2)
points(D[11:n, ], pch = 4, col = 2, lwd = 2, cex = 2)

plot(Y, pch = NA, main = "Output Space",
     xlab = expression(y[1]), ylab = expression(y[2]),
     cex.main = 2.5, cex.axis = 1.5, cex.lab = 1.5)
points(Y[1:10, ], pch = 16, col = 4, cex = 2)
points(Y[11:n, ], pch = 4, col = 2, lwd = 2, cex = 2)
```




```
par(mfrow = c(1, 1))
```

6 Summary

This chapter covered:

1. **Sequential design for emulation** - Adding points to improve prediction
2. **Expected Improvement (EI)** - Criterion for Bayesian optimization
3. **Active Learning** - Adaptive sampling strategies
   - ALC (Active Learning Cohn)
   - ALMV (Active Learning MacKay-Vivarelli)
   - ALM (Active Learning MacKay)
4. **Inverse designs (OSFD)** - Space-filling in output space